

Extraktion und Verwaltung prozeduralen Wissens aus geschriebener Kommunikation

Practical Seminar [T-WIWI-114892]

Adrian Noah Wimmer
2308067

At the Department of Economics and Management
Institute for Information Systems (WIN)
Information Systems I | human-centered systems lab (h-lab)

Reviewer: Prof. Dr. Alexander Maedche
Supervisor: Simon Rapp

30.09.2026

Inhaltsverzeichnis

1	Einleitung	1
2	Grundlagen	1
3	Verwandte Arbeiten	1
4	Konzept	2
5	Umsetzung	4
5.1	Aufbereitung	4
5.2	Gruppierung	4
5.3	Extraktion	5
5.4	Weboberfläche	6
6	Evaluation	6
6.1	Gruppierung	6
6.2	Korrektheit der Ketten	7
6.3	Kompaktheit	7
6.4	Erhalt von Schritten und Abhängigkeiten	8
6.5	Einschränkungen	8
7	Fazit und Ausblick	8
	Literatur	9

1 Einleitung

In Organisationen ergibt sich ein großer Teil des prozeduralen Wissens implizit aus geschriebener Kommunikation, etwa durch E-Mails, Transkripte oder Chatverläufe. Darin sind häufig wiederkehrende Abläufe, Zuständigkeiten, Entscheidungen und konkrete Handlungsschritte, wenn auch implizit, enthalten. In ihrer ursprünglichen Form sind diese Informationen jedoch oft unübersichtlich, redundant und schwer explizit auffindbar.

Eine mögliche Lösung besteht darin, aus solchen Kommunikationsdaten strukturierte und standardisierte Aktionsketten abzuleiten. Diese können als vereinfachte Prozessdarstellungen umgesetzt werden, um prozedurales Wissen explizit zu machen. Anstatt relevante (Meta-)Informationen direkt aus vielen einzelnen E-Mails oder Transkripten suchen zu müssen, könnten Nutzer auf kompaktere und besser strukturierte Repräsentationen zugreifen.

Eine zentrale Herausforderung liegt dabei in der Zuordnung von Nachrichten zu Prozessen. In der Praxis ist ein Prozess selten in einer einzelnen E-Mail abgebildet, sondern erstreckt sich über mehrere Nachrichten, teils mit völlig unterschiedlichen Betreffzeilen, und ist mit Nachrichten zu anderen Vorgängen vermischt. Bestehende Ansätze zur Prozessextraktion aus Text setzen hingegen überwiegend voraus, dass ein Dokument genau einen Prozess beschreibt ([1, 2]).

In dieser Arbeit wird eine Anwendung konzipiert und prototypisch umgesetzt, die implizites prozedurales Wissen aus E-Mail-Kommunikation extrahiert und in Form strukturierter Aktionsketten zugänglich macht. Als Datengrundlage dient das Enron-E-Mail-Korpus ([3]). Eine Python-basierte Pipeline leitet daraus mithilfe eines Large Language Models (LLM) Aktionsketten ab, die anschließend in einer Next.js-Webanwendung¹ als Prozessgraph visualisiert und durchsuchbar gemacht werden. Im Fokus steht dabei die Frage, wie zusammengehörige E-Mails in einem ungefilterten Postfach identifiziert und zu einer gemeinsamen Aktionskette zusammengeführt werden können.

2 Grundlagen

Das Practical Seminar baut auf dem Konzept des prozeduralen Wissens auf. Dies ist das praktisch anwendbare Handlungswissen darüber, wie man bestimmte Tätigkeiten oder Prozeduren ausführt ([4]). Durch die Ableitung von Aktionsketten soll dieses oft implizite Wissen in eine explizite und strukturierte Form überführt werden ([5]). Aktionsketten beschreiben Abläufe als Folge einzelner Schritte und können sich an vereinfachten Prozessmodellen orientieren ([6]). KI-basierte Verfahren der Sprachverarbeitung können dabei helfen, relevante Informationen aus den Kommunikationsdaten zu extrahieren und in einem strukturierten Format für Suche, Darstellung und Bearbeitung nutzbar zu machen ([7]).

3 Verwandte Arbeiten

Die automatische Extraktion von Prozessmodellen aus natürlichsprachlichem Text ist seit über einem Jahrzehnt Gegenstand der Forschung zum Business Process Management (BPM).

¹<https://nextjs.org/>

Sie gilt als anspruchsvoll, da Sprache mehrdeutig ist und Schreibstile sowie Domänen stark variieren [8]. Ihre Relevanz ergibt sich daraus, dass prozessrelevantes Wissen in Organisationen häufig nur implizit vorliegt [9].

Als grundlegend gilt ein regelbasierter Ansatz aus dem Jahr 2011 [1]. Er ist dadurch motiviert, dass die Erstellung von Ist-Modellen bis zu 60 % der Zeit in Prozessprojekten beansprucht, obwohl die nötige Information oft bereits als Dokumentation vorliegt. Die Autoren kombinieren etablierte NLP-Werkzeuge, darunter den Stanford Parser und WordNet, mit einer eigenen Anaphernauflösung. Den Text überführen sie zunächst in eine Zwischenrepräsentation aus Akteuren, Ressourcen, Aktionen und Flüssen, aus der das BPMN-Modell erzeugt wird. Auf 47 Text-Modell-Paaren aus Industrie und Lehrbüchern werden im Mittel 77 % der Modelle korrekt generiert.

Mit PET entstand der erste annotierte Referenzkorpus des Feldes [2]. Er umfasst 45 Prozessbeschreibungen auf Basis der Texte aus [1] mit im Mittel rund neun Sätzen je Dokument, die von jeweils drei Experten annotiert wurden. Die mitgelieferten Baselines erreichen hohe Precision bei niedrigem Recall. Schwierig ist also vor allem, alle Elemente zu finden. Ein Vergleich von zehn seit 2010 veröffentlichten Ansätzen zeigt, dass diese überwiegend regelbasiert sind und neuere NLP-Verfahren kaum nutzen [8]. Neuere Arbeiten setzen dagegen Sprachmodelle ein [7]: Ein feinjustiertes BERT-Modell extrahiert die Prozesselemente, GPT-3.5-Turbo bzw. GPT-4 erkennt die Gateways. Auf 31 Beschreibungen erreicht die Pipeline mit GPT-4 eine Genauigkeit von 96 %, mit GPT-3.5-Turbo 80 %.

Die betrachteten Arbeiten gehen überwiegend von einem zusammenhängenden Dokument aus, das genau einen Prozess beschreibt. Nur dann lässt sich die Satzreihenfolge als Ablaufreihenfolge lesen. Die Referenzdaten enthalten zudem kaum prozessirrelevante Passagen, weshalb deren Herausfiltern bislang selten betrachtet wurde [8]. Auch bei der LLM-basierten Pipeline wird eingeräumt, dass reale Beschreibungen weniger klar formuliert sein können als die Testdaten [7]. Eine Ausnahme bildet ein Ansatz, der von Beteiligten erzählte Prozessgeschichten vor der Extraktion nach Prozess gruppiert [10]. Seine Eingaben sind jedoch gezielt verfasste Beschreibungen eines Ablaufs und kein ungefiltertes Postfach. In E-Mails verteilt sich ein Prozess über mehrere Nachrichten, die mit irrelevanten Inhalten und anderen Vorgängen vermischt sind. Vor der Extraktion muss daher bestimmt werden, welche Nachrichten zusammengehören. An diesem Schritt setzt die vorliegende Arbeit an.

4 Konzept

Aus der in Abschnitt 3 beschriebenen Lücke ergeben sich vier Anforderungen. (A1) Die Anzahl der Prozesse in einem Postfach ist unbekannt und wird nicht vorgegeben. (A2) Es erfolgt keine Vorauswahl nach Stichworten, da diese bereits festlegen würde, welche Prozesse gesucht werden. (A3) Das System muss ein leeres Ergebnis liefern können, statt zusammenhanglose Nachrichten zu einer Kette zu verbinden. (A4) Jeder extrahierte Schritt muss auf seine Quellnachricht zurückführbar sein.

Die zentrale Entwurfsentscheidung ist die Zerlegung in eine Gruppierungs- und eine Extraktionsstufe. Bestehende Verfahren beginnen direkt mit der Extraktion, weil ihre Eingabe

bereits genau einen Prozess enthält. Bei einem ungefilterten Postfach muss zuerst geklärt werden, welche Nachrichten überhaupt zusammengehören.

Gruppierung.

Aus A1 folgt ein dichte-basiertes Verfahren, bei dem sich die Anzahl der Gruppen aus den Daten ergibt und Nachrichten ohne Prozessbezug als Rauschen keiner Gruppe zugeordnet werden. Gruppierung wird nach semantischer Ähnlichkeit statt nach Betreff oder Thread-Kennung, da sich ein Vorgang über Nachrichten mit unterschiedlichen Betreffzeilen erstrecken kann. Die Gruppierung wird nicht als fehlerfrei vorausgesetzt. Ob eine Gruppe tatsächlich einen Prozess enthält, prüft erst die Extraktionsstufe, die den Text ohnehin liest.

Extraktion.

Jede Gruppe wird von einem LLM verarbeitet, das anders als die regelbasierten Pipelines aus Abschnitt 3 auch verrauschte und unvollständige Formulierungen verarbeiten kann. Gegen plausible, aber unbelegte Abläufe gelten drei Festlegungen: Jeder Aufgabenknoten benötigt ein wörtliches Belegzitat (A4). Kanten entstehen nur, wenn ein Schritt inhaltlich auf einem anderen aufbaut, nicht bei bloßer zeitlicher Abfolge. Ob überhaupt ein Prozess vorliegt, wird vor der Extraktion geprüft, sodass ein leeres Ergebnis zulässig ist (A3). Eine Gruppe kann außerdem mehrere Ketten ergeben, da eine Nachricht mehrere parallele Vorgänge betreffen kann. Aus demselben Grund werden Rausch-Nachrichten nicht verworfen, sondern einzeln geprüft.

Datenschema.

Das Schema der Aktionsketten orientiert sich an PET [2]. Das vermeidet eine willkürliche Eigenentwicklung und hält die Ergebnisse an bestehende Arbeiten anschlussfähig (Tabelle 1). PET annotiert Textspannen innerhalb eines Dokuments. Da eine Aktionskette Inhalte mehrerer E-Mails zusammenführt, werden die Elemente hier als eigenständige Objekte modelliert und über **sources** und **evidence** an Quellnachricht und wörtliches Zitat zurückgebunden. Diese Felder ersetzen die Spannenannotation von PET und setzen zugleich Anforderung A4 um. Auch das Vorgehen, die Extraktion gegen eine manuelle Referenz zu messen, ist an PET angelehnt (Abschnitt 6).

Darstellung.

Die Ketten werden in einer Webanwendung durchsuchbar gemacht und als Ablaufdiagramm dargestellt. Zu jedem Schritt werden Quellnachricht und Belegzitat angezeigt, sodass die Herkunft für Nutzer nachvollziehbar bleibt. Da die Extraktion fehlerbehaftet ist, lassen sich Ketten zudem manuell anlegen, bearbeiten und löschen.

Tabelle 1: Zuordnung der PET-Elemente zum Schema der Aktionsketten

PET	Aktionskette
Activity	Knoten vom Typ <code>task</code>
Actor	<code>actors</code>
Actor Performer / Recipient	<code>actor</code> / <code>recipient</code> am Knoten
Activity Data, Uses	<code>inputs</code> / <code>outputs</code> am Knoten
Gateway (XOR, AND)	Knoten vom Typ <code>gateway</code> (<code>xor</code> , <code>and</code> , <code>or</code>)
Condition Specification	<code>condition</code> an der Kante
Flow	<code>flows</code>
Same Gateway	entfällt, da ein Gateway ein Knoten ist
Further Specification	nicht übernommen
–	<code>sources</code> , <code>evidence</code> , <code>prozess_wahrscheinlichkeit</code>

5 Umsetzung

Das System besteht aus einer Python-Pipeline und einer Next.js-Webanwendung, die über JSON-Dateien gekoppelt sind (Abbildung 1).² Der Enron-Korpus wird über die Kommandozeile verarbeitet, Postfächer können zusätzlich über die Weboberfläche hochgeladen werden. Beide Wege erzeugen dasselbe Zwischenformat. Jeder Upload wird als unabhängiger Batch verarbeitet und verändert bestehende Ketten nicht.

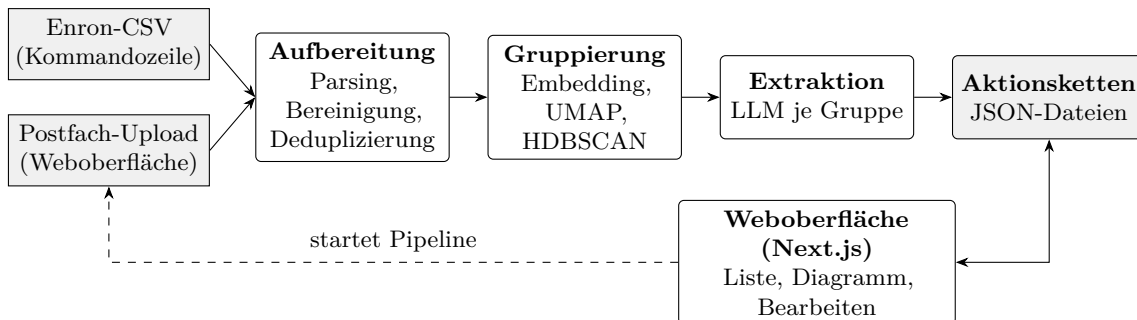


Abbildung 1: Architektur des Systems. Beide Eingangswege erzeugen dasselbe Zwischenformat. Die Weboberfläche liest und schreibt die Ergebnisd Dateien und startet bei einem Upload die Pipeline als Hintergrundprozess.

5.1 Aufbereitung

Uploads werden als `.eml`, `.mbox`, `.pst` oder `.zip` akzeptiert, `.pst`-Dateien liest ein Node.js-Skript mit `pst-extractor`. Beide Eingangswege nutzen dieselbe Bereinigung: Zitate und weitergeleitete Vorgängernachrichten werden entfernt, Betreffpräfixe normalisiert, HTML in Text umgewandelt und Nachrichten unter 40 Zeichen verworfen. Da der Enron-Korpus dieselbe Nachricht in mehreren Postfächern enthält, wird anschließend exakt über den normalisierten Text dedupliziert.

5.2 Gruppierung

Die Gruppierung erfolgt in drei Schritten. Zunächst werden Betreff und die ersten 6000 Zeichen jeder Nachricht mit `kit.qwen3-embedding-8b` über die KIT-Toolbox³ in einen

²Quellcode: https://github.com/adriannoahwimmer/Extraktion_Aktionsketten_aus_Emails/tree/abgabe-2026-09-30

³<https://ki-toolbox.scc.kit.edu>

4096-dimensionalen Vektor eingebettet. Die Vektoren werden lokal zwischengespeichert. Da sich paarweise Abstände in hochdimensionalen Räumen angleichen und Dichteunterschiede dadurch schwer messbar werden, reduziert UMAP [11] die Vektoren anschließend auf fünf Dimensionen (Kosinusmetrik, 15 Nachbarn, fester Startwert). Bei weniger als 15 Nachrichten entfällt dieser Schritt. Abschließend weist HDBSCAN [12] jede Nachricht einem Cluster zu. Die Anzahl der Cluster ergibt sich dabei aus der Dichtestruktur, Nachrichten in Bereichen geringer Dichte werden als Rauschen markiert. Verwendet wird die Implementierung aus `scikit-learn` mit euklidischer Metrik auf den UMAP-Koordinaten. Die minimale Clustergröße ist 2, da reale Vorgänge oft nur zwei bis drei Nachrichten umfassen. Die Wahl beruht auf einem Parameter-Sweep (Abschnitt 6).

5.3 Extraktion

Auswahl und Modell.

Jedes Cluster bildet eine Gruppe. Im vollständigen Lauf und bei Uploads kommt jede Rausch-Nachricht mit mindestens 200 Zeichen, die nicht von einem Massenversender stammt, als Einzelgruppe hinzu. Ausgewählt werden die größten Gruppen, begrenzt auf 300 LLM-Aufrufe je Lauf. Die Nachrichten einer Gruppe werden chronologisch mit Absender, Empfänger, Datum und Betreff an `google.claude-sonnet-5` übergeben, im Upload alternativ an `google.claude-opus-4.8`, jeweils über den OpenAI-kompatiblen Endpunkt der KIT-Toolbox mit maximal 16 000 Ausgabebtokens. Da das Modell `temperature` nicht unterstützt, sind die Läufe nicht deterministisch.

Prompt.

Der Systemprompt enthält die Definition von prozeduralem Wissen und Aktionskette, die Arbeitsreihenfolge aus Abschnitt 4 (Prozesse identifizieren und trennen, bewerten, ausgeben), das Ausgabeschema und Qualitätsregeln. Formal ist eine Aktionskette ein gerichteter Graph $C = (V_T \cup V_G, E, A, S)$ mit Aufgabenknoten V_T , Gateways V_G (`xor`, `and`, `or`), Kanten E mit optionaler Bedingung, Akteuren A und Quellnachrichten S . Jedem $v \in V_T$ sind ein ausführender Akteur und eine nichtleere Belegmenge $\varepsilon(v)$ aus Quelle und wörtlichem Zitat zugeordnet. Hinzu kommt die Selbsteinschätzung $p(C) \in [0, 1]$ mit Begründung. Über die Festlegungen aus Abschnitt 4 hinaus verlangt der Prompt: Handlungen einer Person nur mit eigener Nachricht als Beleg, gültige Verweise auf Akteure, Quellen und Knoten, mindestens zwei bedingte ausgehende Kanten je Gateway sowie ASCII-Umschreibung deutscher Umlaute, da diese über die Schnittstelle doppelt kodiert zurückkamen. Ausgegeben werden nur Ketten mit $p(C) \geq 0,4$ und $|V_T| \geq 2$. Diese Regeln sind nur im Prompt verankert, ihre Einhaltung prüft Abschnitt 6 nachträglich.

Robustheit.

Vorübergehende API-Fehler werden bis zu viermal mit wachsender Wartezeit wiederholt. Fehlerhafte oder leere Antworten führen nur zum Überspringen der betroffenen Gruppe. Bearbeitete Gruppen werden vermerkt, sodass ein abgebrochener Lauf fortgesetzt werden kann.

5.4 Weboberfläche

Die Weboberfläche liest und schreibt die Ketten direkt als JSON-Dateien. Die Übersicht bietet eine Volltextsuche und kennzeichnet $p(C)$ farblich. Die Detailansicht zeigt das Ablaufdiagramm mit Aufgaben als Rechtecken, Gateways als Rauten und Bedingungen an den Kanten, darunter Akteure und Quellnachrichten mit Belegzitate. Beim Upload legt eine API-Route ein Batch-Verzeichnis an und startet die Pipeline als Hintergrundprozess, deren Fortschritt das Frontend alle zwei Sekunden abfragt. Tabelle 2 fasst die verwendeten Technologien zusammen. Die Entwicklung erfolgte mit Unterstützung von Claude Code (Anthropic).

Tabelle 2: Verwendete Technologien

Bereich	Komponenten (Version)
Pipeline	Python 3.13, openai 2.46, numpy 2.5, scikit-learn 1.9, umap-learn 0.5
.pst-Import	Node.js 24, pst-extractor 1.12
Weboberfläche	Next.js 16.3, React 19.2, TypeScript 5, Tailwind CSS 4
Diagramm	React Flow (@xyflow/react) 12, dagre 3
Modelle	kit.qwen3-embedding-8b (Embedding), google.claude-sonnet-5 und google.claude-opus-4.8 (Extraktion)

6 Evaluation

Die Evaluation prüft drei Fragen: wie viele Aktionsketten korrekt extrahiert werden, wie stark sie die Kommunikation verdichten und ob relevante Handlungsschritte und Abhängigkeiten erhalten bleiben. Entwicklung und Parameterwahl erfolgten auf den ersten 2000 Zeilen der Enron-CSV-Datei (750 Nachrichten nach Deduplizierung), der Hauptlauf umfasst die ersten 30 000 Zeilen (Tabelle 3).⁴

Als Referenzfall dient ein Immobilienverkauf, der im Korpus unter dem Stichwort „Burnet“ geführt wird. Vor dem Verkauf müssen ein Lageplan vorliegen und eine schriftliche Zustimmung erteilt werden. Die zugehörigen Nachrichten tragen unterschiedliche Betreffzeilen, sodass der Vorgang nur über den Inhalt als zusammengehörig erkennbar ist. Für diesen Vorgang wurde eine Referenzkette manuell annotiert.

6.1 Gruppierung

Die Deduplizierung entfernt mehr als die Hälfte der Nachrichten. Knapp die Hälfte der Cluster umfasst genau zwei Nachrichten, was die minimale Clustergröße von 2 stützt. Ein Parameter-Sweep über elf Kombinationen aus Nachbarzahl $k \in \{10, 15, 30, 50\}$ und minimaler Clustergröße $\in \{2, 3, 4\}$ zeigte, dass größere Mindestgrößen schwache Cluster nicht ins Rauschen verschieben, sondern zu größeren Fehlclustern verschmelzen.

Auf dem Entwicklungsausschnitt lagen die Burnet-Nachrichten in acht der elf Konfigurationen, darunter der gewählten, im selben Cluster. Im Hauptlauf mit rund 15-mal so

⁴Verwendet wurden die zweite Prompt-Version, belegt über das Ausgabeformat, und `claude-sonnet-5`. Das Modell speichern die Ketten nicht; es ergibt sich aus dem Standardwert im Code und den Protokollen.

Tabelle 3: Kennzahlen des Hauptlaufs (erste 30 000 CSV-Zeilen)

<i>Aufbereitung und Gruppierung</i>		
Nachrichten nach Bereinigung		24 813
nach Deduplizierung	11 627 (53 % Dubletten)	
Cluster		2 710
Rauschen	2 656 (22,8 %)	
Clustergröße (Median / Maximum)		3 / 29
Cluster mit genau zwei Nachrichten		1 283
<i>Extraktion</i>		
verarbeitete Gruppen	300 ausgewählt, ca. 54 beantwortet	
extrahierte Ketten	51 aus 36 Clustern	
Cluster mit mehr als einer Kette		13
$p(C)$ (Median / Spanne)		0,65 / 0,4–0,9
Knoten je Kette (Median / Spanne)		5 / 2–12
Ketten mit Gateway		9
Belegzitate wörtlich in der Quelle	245 von 252 (97,2 %)	
ungültige Verweise		0
<i>Tokens Kette / Quellnachrichten (Median)</i>		
lesbarer Teil (Titel, Zusammenfassung, Aktionen)		0,39
vollständiges JSON		2,04

vielen Nachrichten verteilen sie sich dagegen auf drei Cluster. Die auf dem Ausschnitt gewählten Parameter übertragen sich also nicht vollständig auf größere Datenmengen, und die Zusammenführung von Nachrichten mit unterschiedlichen Betreffzeilen gelingt nicht zuverlässig.

6.2 Korrektheit der Ketten

Von den 252 Belegzitate stehen 245 (97,2 %) wörtlich in der angegebenen Quellnachricht. Bei fünf der sieben Abweichungen ist das Zitat inhaltlich korrekt, aber leicht verändert, gekürzt oder durch Kodierungsartefakte gestört. Bei zwei verweist die Quellenangabe auf die falsche Nachricht desselben Clusters. Alle Verweise auf Akteure, Quellen und Knoten sind gültig.

Inhaltlich wurde eine Zufallsstichprobe von zehn der 51 Ketten anhand der Quellnachrichten und ohne Kenntnis von $p(C)$ bewertet. Als korrekt gilt eine Kette, wenn alle Schritte durch die Quellen gedeckt sind und keine Kante dem Ablauf widerspricht. Alle zehn Ketten sind korrekt (95-%-Konfidenzintervall nach Clopper-Pearson: 69–100 %). Sieben beschreiben ein übertragbares Verfahren, drei dagegen einmalige Vorgänge wie die Organisation einer Veranstaltung für Praktikanten. Formal korrekte Ketten enthalten also nicht zwangsläufig übertragbares prozedurales Wissen. Die Selbsteinschätzung trennt die beiden Gruppen nicht: Übertragbare wie einmalige Ketten erreichen im Median $p(C) = 0,75$. Sie unterscheidet damit nicht zwischen übertragbarem Wissen und einmaligen Vorgängen.

6.3 Kompaktheit

Der lesbare Teil einer Kette umfasst im Median 39 % der Tokens ihrer Quellnachrichten, in Summe 19 %.⁵ Das vollständige JSON ist dagegen im Median doppelt so lang wie die

⁵Gezählt mit dem Tokenizer `c1100k_base` als Näherung.

Quellen, da Belegzitate, Quellenangaben und Metadaten den Text teilweise wiederholen. Die Verdichtung betrifft somit die Darstellung für Nutzer, nicht das Speicherformat.

6.4 Erhalt von Schritten und Abhängigkeiten

Da die Burnet-Cluster nicht unter den verarbeiteten Gruppen lagen, wurden sie für den Vergleich mit der Referenzkette gesondert extrahiert. Eine automatische Zuordnung über Textähnlichkeit findet für zwei der fünf Referenzschritte eine Entsprechung und für keine der vier Referenzkanten. Die schriftliche Freigabe, von der der Verkauf abhängt, erscheint nicht als eigener Schritt, sondern nur als Bedingung an einer Kante. Die Abhängigkeit bleibt damit inhaltlich erhalten, ist aber anders modelliert als in der Referenz. Ursache der Lücken ist vor allem die Gruppierung: Da die Nachrichten des Vorgangs auf mehrere Cluster verteilt sind, sieht das Modell ihn nie vollständig. Über alle Ketten enthalten nur 9 von 51 ein Gateway. Vermutlich dokumentieren E-Mails meist nur den tatsächlich eingeschlagenen Weg, sodass Bedingungen und Alternativen selten erhalten bleiben.

6.5 Einschränkungen

Der Hauptlauf endete nach etwa 54 der 300 ausgewählten Gruppen, weil die KIT-Toolbox weitere Anfragen ablehnte, wegen aufgebrauchten Guthabens. Die 51 Ketten stammen daher aus einer Teilmenge der größten Cluster. Die Clusterparameter wurden auf einem deutlich kleineren Ausschnitt gewählt. Die inhaltliche Bewertung beruht auf einer Stichprobe von zehn Ketten, einem Referenzfall und einem Bewerter. Die Läufe sind nicht deterministisch.

7 Fazit und Ausblick

Diese Arbeit zeigt, dass sich aus einem ungefilterten Postfach ohne Vorgabe der Prozessanzahl belegte Aktionsketten extrahieren lassen. In der bewerteten Stichprobe sind alle Ketten durch ihre Quellen gedeckt, und die lesbare Darstellung verdichtet die zugrunde liegende Kommunikation deutlich. Die Zusammenführung von Nachrichten mit unterschiedlichen Betreffzeilen aber mit Zusammenhang, gelingt auf größeren Datenmengen jedoch nicht zuverlässig, und nicht jede korrekte Kette enthält übertragbares prozedurales Wissen. Daraus ergeben sich drei Richtungen für die Weiterentwicklung.

Erstens beschreiben die Ketten zunächst nur, wie ein bestimmter Vorgang abgelaufen ist, nicht, wie er im Allgemeinen durchgeführt wird. Drei der zehn bewerteten Ketten betreffen einmalige Vorgänge wie die Organisation einer Veranstaltung, und die Selbsteinschätzung des Modells unterscheidet sie nicht von übertragbaren. Innerhalb einer einzelnen Gruppe kann das Modell Wiederholung auch nicht erkennen. Eine zusätzliche Aggregationsstufe könnte Ketten desselben Prozesstyps gruppenübergreifend zu einer verallgemeinerten Kette zusammenführen, ähnlich wie Process Mining ein Modell aus vielen Ausführungsinstanzen ableitet [13]. Erst die Unterschiede zwischen den Instanzen würden dabei auch die Verzweigungen sichtbar machen, die in einzelnen Vorgängen kaum auftreten.

Zweitens werden bisher Nachrichten zu Ketten kombiniert, aber keine Ketten untereinander. Ketten aus verschiedenen Gruppen oder Uploads bleiben getrennt, überlappende Postfächer

erzeugen doppelte Ketten, und neue Nachrichten ergänzen bestehende Ketten nicht. Eine Weiterentwicklung würde neue Nachrichten und Ketten vorhandenen Ketten zuordnen, etwa über die Ähnlichkeit ihrer Quellnachrichten, und diese ergänzen oder versionieren. Dabei wäre zu klären, wie manuelle Änderungen gegenüber automatischen Aktualisierungen geschützt werden.

Drittens sind Gruppierung und Evaluation auszubauen. Die Clusterparameter sind auf größeren Datenmengen neu zu bestimmen, und Verfahren mit überlappender Zuordnung würden berücksichtigen, dass eine Nachricht mehrere Vorgänge betreffen kann. Die Evaluation sollte einen vollständigen Lauf über alle Cluster und Einzelnachrichten, weitere handannotierte Referenzvorgänge nach dem Vorgehen von PET [2], mehrere unabhängige Bewerter sowie die Eignung für Suche und Retrieval umfassen.

Literatur

- [1] Fabian Friedrich, Jan Mendling und Frank Puhlmann. „Process model generation from natural language text“. In: *Advanced Information Systems Engineering: 23rd International Conference, CAiSE 2011*. Bd. 6741. Lecture Notes in Computer Science. Springer, 2011, S. 482–496. DOI: 10.1007/978-3-642-21640-4_36.
- [2] Patrizio Bellan u. a. „PET: An Annotated Dataset for Process Extraction from Natural Language Text Tasks“. In: *Business Process Management Workshops: BPM 2022 International Workshops*. Bd. 460. Lecture Notes in Business Information Processing. Springer, 2022, S. 315–321. DOI: 10.1007/978-3-031-25383-6_23.
- [3] William Cukierski. *The Enron Email Dataset*. Kaggle, 2016. URL: <https://www.kaggle.com/datasets/wcukierski/enron-email-dataset> (besucht am 22.09.2026).
- [4] Jason Stanley und Timothy and Willamson. „Knowing How“. In: *Journal of Philosophy* 98.8 (2001), S. 411–444. ISSN: 0022-362X. DOI: 10.2307/2678403. URL: <http://dx.doi.org/10.2307/2678403>.
- [5] Ikujiro Nonaka, Ryoko Toyama und Noboru Konno. „SECI, Ba and Leadership: a Unified Model of Dynamic Knowledge Creation“. In: *Long Range Planning* 33.1 (Feb. 2000), S. 5–34. ISSN: 0024-6301. DOI: 10.1016/S0024-6301(99)00115-6. URL: [http://dx.doi.org/10.1016/S0024-6301\(99\)00115-6](http://dx.doi.org/10.1016/S0024-6301(99)00115-6).
- [6] Albert T. Corbett und John R. Anderson. „Knowledge tracing: Modeling the acquisition of procedural knowledge“. In: *User Modelling and User-Adapted Interaction* 4.4 (1995), S. 253–278. ISSN: 1573-1391. DOI: 10.1007/bf01099821. URL: <http://dx.doi.org/10.1007/BF01099821>.
- [7] Josip Tomo Licardo, Nikola Tanković und Darko Etinger. „A Method for Extracting BPMN Models from Textual Descriptions Using Natural Language Processing“. In: *Procedia Computer Science* 239 (2024), S. 483–490. DOI: 10.1016/j.procs.2024.06.196.
- [8] Patrizio Bellan u. a. *Process Extraction from Text: Benchmarking the State of the Art and Paving the Way for Future Challenges*. Version 2. 2023. DOI: 10.48550/arXiv.2110.03754. arXiv: 2110.03754.
- [9] Ikujiro Nonaka, Ryoko Toyama und Noboru Konno. „SECI, Ba and leadership: a unified model of dynamic knowledge creation“. In: *Long range planning* 33.1 (2000), S. 5–34.

- [10] J. C. de A. R. Gonçalves, F. M. Santoro und F. A. Baião. „Let Me Tell You a Story – On How to Build Process Models“. In: *Journal of Universal Computer Science* 17.2 (2011), S. 276–295. DOI: 10.3217/jucs-017-02-0276.
- [11] Leland McInnes, John Healy und James Melville. *UMAP: Uniform manifold approximation and projection for dimension reduction*. 2018. DOI: 10.48550/arXiv.1802.03426. arXiv: 1802.03426.
- [12] Ricardo J. G. B. Campello, Davoud Moulavi und Jörg Sander. „Density-based clustering based on hierarchical density estimates“. In: *Advances in Knowledge Discovery and Data Mining: PAKDD 2013*. Bd. 7819. Lecture Notes in Computer Science. Springer, 2013, S. 160–172. DOI: 10.1007/978-3-642-37456-2_14.
- [13] Wil M. P. van der Aalst. *Process Mining: Data Science in Action*. 2. Aufl. Springer, 2016. DOI: 10.1007/978-3-662-49851-4.